IR_Text_Classification_Assignment

Name: Harini K K

Reg No: RA2411026050041

Class: CSE AIML A

Year: 3rd Year

GitHub Repository

https://github.com/harini-kk-1606/IR_Text_Classification_Assignment

Project Overview

This project classifies real technical discussion posts from the 20 Newsgroups dataset into six classes: Cryptography, Electronics, Graphics, PC Hardware, Space and Windows OS. It extends the Task 1 TF-IDF system, which worked on computing-topic documents, from ranking documents to labelling them. Each post is first cleaned by removing its e-mail header, quoted replies and signature. It is then preprocessed with lowercasing, tokenization, stop word removal and Porter stemming, and represented as a TF-IDF vector. Decision Tree, KNN, SVM and Naive Bayes are trained on the same 75/25 stratified split and compared using Accuracy, Precision, Recall, F1-score and confusion matrices. As a what-if experiment, the feature representation was changed between bag-of-words counts, TF-IDF unigrams and TF-IDF unigrams+bigrams.

Code and Execution Screenshots

text_classification.py

```python
"""
IR CLA-2 | Task 2 : Text Classification Battle
Compare Decision Tree, KNN, SVM and Naive Bayes on technical newsgroup posts.

Student : Harini K K (RA2411026050041), III CSE AIML A
Dataset : 20 Newsgroups (Lang, 1995) - six computing / science groups
Usage   : python text_classification.py
"""

import json
import re
import time
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer, ENGLISH_STOP_WORDS
from sklearn.metrics import accuracy_score, confusion_matrix, f1_score, precision_score, recall_score
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import LinearSVC
from sklearn.tree import DecisionTreeClassifier

BASE = Path(__file__).resolve().parent
DATA = BASE / "dataset" / "tech_newsgroups.csv"
OUT = BASE / "outputs"
PLOTS = BASE / "plots"
OUT.mkdir(exist_ok=True)
PLOTS.mkdir(exist_ok=True)

RANDOM_STATE = 7
TEST_FRACTION = 0.25

CLASSES = {
    "comp.graphics": "Graphics",
    "comp.os.ms-windows.misc": "Windows OS",
    "comp.sys.ibm.pc.hardware": "PC Hardware",
    "sci.crypt": "Cryptography",
    "sci.electronics": "Electronics",
    "sci.space": "Space",
}

porter = PorterStemmer()
STOPWORDS = ENGLISH_STOP_WORDS
```

Figure 1: Imports, dataset path and class definitions

text_classification.py

```python
# 1. Cleaning and preprocessing
# ---------------------------------------------------------------------------
def strip_newsgroup_noise(post):
    """Remove parts of a Usenet post that would reveal the group without
    reading the content: the header block (From, Organization, e-mail
    addresses ...), quoted replies and the signature. Only the Subject line
    is kept from the header because it is written by the author."""
    head, _, body = post.partition("\n\n")
    subject = ""
    for line in head.splitlines():
        if line.lower().startswith("subject:"):
            subject = line[8:].replace("Re:", "").strip()
    lines = []
    for line in body.splitlines():
        if line.strip().startswith((">", "|", ":")):
            continue  # quoted text from an earlier message
        if re.search(r"(writes|wrote|says):\s*$", line):
            continue  # "In article <...> someone writes:"
        if line.strip() in ("--", "-- "):
            break  # signature starts here
        lines.append(line)
    return subject + "\n" + "\n".join(lines)


def preprocess(text):
    """Same steps as Task 1: lowercase, alphabetic tokens, stop-word
    removal, Porter stemming."""
    text = re.sub(r"\S+@\S+", " ", text.lower())  # e-mail ids
    text = re.sub(r"http\S+|www\.\S+", " ", text)  # urls
    words = re.findall(r"[a-z]+", text)
    return " ".join(porter.stem(w) for w in words if len(w) > 2 and w not in STOPWORDS)


def load_dataset():
    df = pd.read_csv(DATA)
    df["label"] = df["newsgroup"].map(CLASSES)
    df["body"] = df["text"].apply(strip_newsgroup_noise)
    df["tokens"] = df["body"].apply(preprocess)
    before = len(df)
    df = df[df["tokens"].str.split().str.len() >= 5]  # drop near-empty posts
    df = df.drop_duplicates(subset="tokens").reset_index(drop=True)
    return df, before - len(df)


# ---------------------------------------------------------------------------
# 2. Feature extraction and classifiers
# ---------------------------------------------------------------------------
def tfidf(ngrams=(1, 1)):
    return TfidfVectorizer(ngram_range=ngrams, min_df=3, max_df=0.9, sublinear_tf=True)


def classifiers():
    return {
        "Decision Tree": DecisionTreeClassifier(criterion="gini", random_state=RANDOM_STATE),
        "KNN": KNeighborsClassifier(n_neighbors=10, metric="cosine", weights="distance"),
        "SVM": LinearSVC(C=1.0, random_state=RANDOM_STATE),
        "Naive Bayes": MultinomialNB(alpha=0.1),
    }


def score(name, clf, Xtr, ytr, Xte, yte):
    start = time.perf_counter()
    clf.fit(Xtr, ytr)
    fit_s = time.perf_counter() - start
    start = time.perf_counter()
    pred = clf.predict(Xte)
    pred_s = time.perf_counter() - start
    return pred, {
        "Classifier": name,
        "Accuracy": accuracy_score(yte, pred),
        "Precision": precision_score(yte, pred, average="macro"),
        "Recall": recall_score(yte, pred, average="macro"),
        "F1-score": f1_score(yte, pred, average="macro"),
        "Train accuracy": accuracy_score(ytr, clf.predict(Xtr)),
        "Fit time (s)": round(fit_s, 4),
        "Predict time (s)": round(pred_s, 4),
    }
```

Figure 2: Post cleaning, preprocessing, TF-IDF and the four classifiers

text_classification.py

```
def main():
    report = []

    def out(line=""):
        print(line)
        report.append(str(line))

    out("TEXT CLASSIFICATION BATTLE - 20 Newsgroups (6 technical groups)")
    out("-" * 64)

    df, dropped = load_dataset()
    out(f"Posts after cleaning          : {len(df)}  (removed {dropped} empty / duplicate posts)")
    counts = df["label"].value_counts()
    out("Posts per class:")
    for lab, n in counts.items():
        out(f"   {lab:<13} {n:>4}  ({100 * n / len(df):.1f}%)")
    out(f"Largest / smallest class ratio: {counts.max() / counts.min():.2f}")
    counts.rename("posts").to_csv(OUT / "class_counts.csv", header=True)

    ex = df.iloc[5]
    (OUT / "cleaning_example.txt").write_text(
        f"CLASS: {ex['label']}\n\n--- RAW POST ---\n{ex['text'][:700]}\n\n"
        f"--- AFTER HEADER / QUOTE REMOVAL ---\n{ex['body'][:500]}\n\n"
        f"--- AFTER PREPROCESSING ---\n{ex['tokens'][:400]}\n")

    labels = sorted(df["label"].unique())
    tr_txt, te_txt, ytr, yte = train_test_split(
        df["tokens"], df["label"], test_size=TEST_FRACTION, stratify=df["label"], random_state=RANDOM_STATE)
    out(f"Training posts: {len(tr_txt)}   Test posts: {len(te_txt)}   (stratified 75/25)")

    vec = tfidf()
    Xtr = vec.fit_transform(tr_txt)
    Xte = vec.transform(te_txt)
    out(f"TF-IDF features: {Xtr.shape[1]}")
    out("")

    rows, cms, preds = [], {}, {}
    for name, clf in classifiers().items():
        pred, row = score(name, clf, Xtr, ytr, Xte, yte)
        rows.append(row)
        preds[name] = pred
        cms[name] = confusion_matrix(yte, pred, labels=labels)
        pd.DataFrame(cms[name], index=labels, columns=labels).to_csv(
            OUT / f"confusion_{name.lower().replace(' ', '_')}.csv")

    table = pd.DataFrame(rows)
    table.to_csv(OUT / "comparison_table.csv", index=False)
    out("RESULTS ON TEST SET (macro averages)")
    out(table.round(4).to_string(index=False))

    per_class = pd.DataFrame(
        {n: f1_score(yte, p, average=None, labels=labels) for n, p in preds.items()}, index=labels)
    per_class.to_csv(OUT / "per_class_f1.csv")
    out("\nF1-score per class")
    out(per_class.round(3).to_string())

    best = table.loc[table["F1-score"].idxmax(), "Classifier"]
    out(f"\nBest classifier: {best}")

    # most frequent confusions of the best model
    cm = cms[best].copy()
    np.fill_diagonal(cm, 0)
    pairs = sorted(((cm[i, j], labels[i], labels[j]) for i in range(len(labels))
                    for j in range(len(labels)) if cm[i, j]), reverse=True)[:5]
    out(f"Top confusions for {best} (actual -> predicted):")
    for n, a, p in pairs:
        out(f"   {a} -> {p}: {n}")

    # interpretable terms
    feats = np.array(vec.get_feature_names_out())
```

Figure 3: Main experiment – split, training, evaluation and confusion analysis

text_classification.py

```python
    # ---------------------------------------------------------------------
    # 5. What-if experiment: change the feature representation
    # ---------------------------------------------------------------------
    out("\nWHAT-IF: same split, same models, different feature representation")
    reps = {
        "Bag-of-words counts": CountVectorizer(min_df=3, max_df=0.9),
        "TF-IDF unigrams": tfidf((1, 1)),
        "TF-IDF uni+bigrams": tfidf((1, 2)),
    }
    wrows = []
    for rep, v in reps.items():
        A, B = v.fit_transform(tr_txt), v.transform(te_txt)
        for name, clf in classifiers().items():
            _, r = score(name, clf, A, ytr, B, yte)
            wrows.append({"Representation": rep, "Features": A.shape[1], "Classifier": name,
                          "Accuracy": r["Accuracy"], "F1-score": r["F1-score"]})
    whatif = pd.DataFrame(wrows)
    whatif.to_csv(OUT / "whatif_feature_representation.csv", index=False)
    out(whatif.pivot(index="Classifier", columns="Representation", values="F1-score").round(4).to_string())
    out("Feature counts: " + ", ".join(
        f"{r} = {n}" for r, n in whatif.groupby("Representation", sort=False)["Features"].first().items()))
    plot_whatif(whatif)

    (OUT / "summary.json").write_text(json.dumps(
        {"posts": int(len(df)), "train": int(len(tr_txt)), "test": int(len(te_txt)),
```

Figure 4: What-if experiment on feature representation

Output — python3 text_classification.py

```
TEXT CLASSIFICATION BATTLE - 20 Newsgroups (6 technical groups)
----------------------------------------------------------------------
Posts after cleaning          : 3507  (removed 37 empty / duplicate posts)
Posts per class:
   Cryptography   595  (17.0%)
   Space          591  (16.9%)
   Electronics    587  (16.7%)
   PC Hardware    581  (16.6%)
   Windows OS     579  (16.5%)
   Graphics       574  (16.4%)
Largest / smallest class ratio: 1.04
Training posts: 2630   Test posts: 877   (stratified 75/25)
TF-IDF features: 6218

RESULTS ON TEST SET (macro averages)
   Classifier  Accuracy  Precision  Recall  F1-score  Train accuracy  Fit time (s)  Predict time (s)
Decision Tree    0.6511     0.6543  0.6500    0.6518          1.0000        0.3158            0.0008
          KNN    0.8609     0.8655  0.8601    0.8614          1.0000        0.0024            0.0359
          SVM    0.8712     0.8724  0.8705    0.8710          0.9992        0.0352            0.0005
  Naive Bayes    0.8746     0.8774  0.8741    0.8742          0.9696        0.0054            0.0005

F1-score per class
             Decision Tree    KNN    SVM  Naive Bayes
Cryptography          0.882  0.966  0.969        0.957
Electronics           0.537  0.862  0.849        0.850
Graphics              0.582  0.841  0.848        0.875
PC Hardware           0.596  0.767  0.803        0.794
Space                 0.714  0.949  0.953        0.944
Windows OS            0.600  0.783  0.804        0.827

Best classifier: Naive Bayes
```

Figure 5: Execution output – dataset, results table, per-class F1 and best model

```
Output — python3 text_classification.py

Top confusions for Naive Bayes (actual -> predicted):
   Windows OS -> PC Hardware: 17
   Electronics -> PC Hardware: 13
   PC Hardware -> Windows OS: 11
   Graphics -> PC Hardware: 11
   Windows OS -> Graphics: 8

WHAT-IF: same split, same models, different feature representation
Representation  Bag-of-words counts  TF-IDF uni+bigrams  TF-IDF unigrams
Classifier
Decision Tree               0.6823              0.6640          0.6518
KNN                         0.8127              0.8779          0.8614
Naive Bayes                 0.8062              0.8888          0.8742
SVM                         0.8427              0.8868          0.8710
Feature counts: Bag-of-words counts = 6218, TF-IDF unigrams = 6218, TF-IDF uni+bigrams = 13547
```

Figure 6: Execution output – what-if experiment

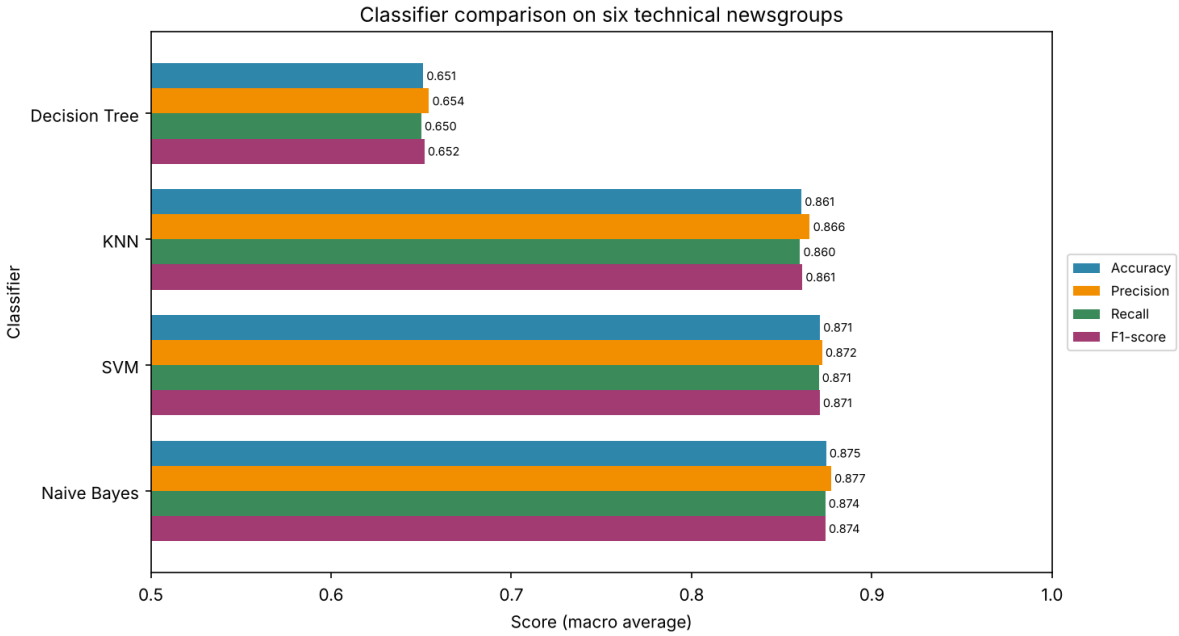


Figure 7: Performance comparison of the four classifiers

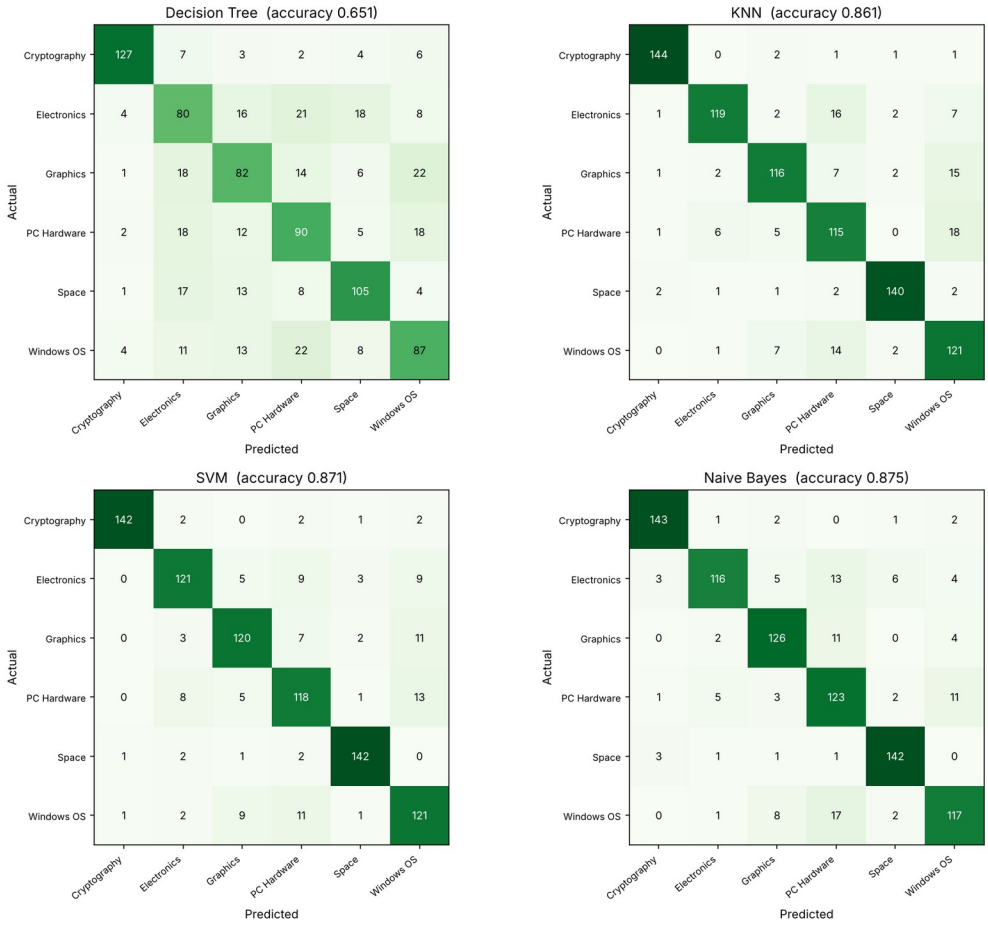


Figure 8: Confusion matrices

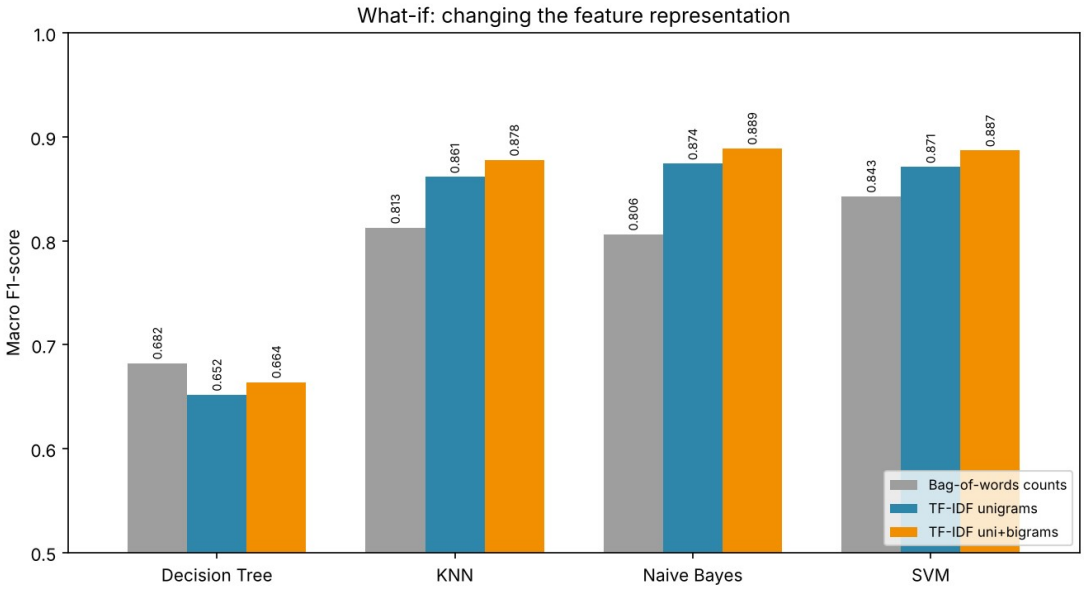


Figure 9: What-if – effect of the feature representation

Project Result

The program processed 3,507 newsgroup posts, built 6,218 TF-IDF features and evaluated the four classifiers on 877 test posts. Naive Bayes gave the best result, with an accuracy of 0.8746 and a macro F1-score of 0.8742. It was closely followed by SVM (0.8710) and KNN (0.8614), while Decision Tree was much lower (0.6518) because it overfits the training data: it scores 100% on training posts. Cryptography and Space were the easiest classes, while most errors came from the closely related Windows OS, PC Hardware and Electronics groups. In the what-if experiment, adding bigrams to TF-IDF improved every model except Decision Tree, and raised Naive Bayes to the highest score of 0.8888. Plain word counts gave the lowest scores for KNN, SVM and Naive Bayes. Naive Bayes with TF-IDF unigram and bigram features is therefore recommended for this task.